

2D Proportional Steering Controller

Linear & Angular Velocity — A Worked Example Guide

Overview

This guide explains how to move a cat toward a mouse in 2D using two values calculated each frame:

Value	What it controls
Linear Velocity (v)	How fast the cat moves forward (units/second)
Angular Velocity (ω)	How fast the cat rotates to face the mouse (degrees/second)

💡 Both values are recalculated every frame using the cat's current position and the mouse's position.

The 5-Step Process

Each frame, repeat these steps:

Step	What happens
Step 1	Calculate vector from cat to mouse (dx, dy, distance)
Step 2	Calculate the desired heading angle using atan2
Step 3	Calculate angle error (how far off the cat is facing)
Step 4	Compute angular velocity ω and linear velocity v
Step 5	Apply velocities to update position and heading

Worked Example

Cat at (2, 3), facing 0° . Mouse at (4, 1).

Step 1 — Vector to Target

Find how far apart they are in x and y, then the straight-line distance:

```
dx      = 4 - 2      = 2
dy      = 1 - 3      = -2
distance = sqrt(22 + 22) = sqrt(8) ≈ 2.83
```

💡 *dx and dy tell us direction. distance tells us how far away the mouse is.*

Step 2 — Desired Angle

Use atan2 to convert the direction vector into an angle:

```
desired_angle = atan2(dy, dx) = atan2(-2, 2) = -45°
```

The mouse is down and to the right, so the cat needs to face -45° (south-east).

Step 3 — Angle Error

How many degrees does the cat still need to turn?

```
angle_error = desired_angle - current_heading
angle_error = -45° - 0° = -45°
```

Negative means turn clockwise (right). Positive means turn anti-clockwise (left).

💡 *Always wrap the result to [-180°, 180°] so the cat takes the shortest turn. Use:*
angle_error = atan2(sin(angle_error), cos(angle_error))

Step 4 — Compute Velocities

Using k_angular = 2.0 and k_linear = 1.5:

Angular Velocity ω

```
 $\omega$  = k_angular × angle_error
 $\omega$  = 2.0 × -45° = -90°/s (turn right at 90 degrees per second)
```

Linear Velocity v

```
v = k_linear × distance × cos(angle_error)
v = 1.5 × 2.83 × cos(-45°)
```

```
v = 1.5 × 2.83 × 0.707
v ≈ 3.0 units/s
```

The $\cos(\text{angle_error})$ term scales speed based on how well-aligned the cat is:

angle_error	Effect on linear speed
0°	$\cos(0^\circ) = 1.0 \rightarrow$ full speed ahead
45°	$\cos(45^\circ) = 0.707 \rightarrow$ 70% speed
90°	$\cos(90^\circ) = 0 \rightarrow$ stop moving forward
180°	$\cos(180^\circ) = -1.0 \rightarrow$ move backwards

💡 This naturally stops the cat from charging in the wrong direction while it is still turning.

Step 5 — Apply Velocities (dt = 0.016s at 60fps)

dt is the time elapsed for one frame. At 60fps: $dt = 1/60 \approx 0.016$ seconds.

Update heading angle

```
θ += ω × dt
θ = 0° + (-90°/s × 0.016s) = -1.44°
```

Update position

```
cx += v × cos(θ) × dt
cy += v × sin(θ) × dt

new cx = 2 + 3.0 × cos(-1.44°) × 0.016 ≈ 2.048
new cy = 3 + 3.0 × sin(-1.44°) × 0.016 ≈ 2.999
```

$\cos(\theta)$ and $\sin(\theta)$ decompose the forward movement into x and y components:

Formula	Meaning
cx += v × cos(θ) × dt	How far the cat moves in x this frame
cy += v × sin(θ) × dt	How far the cat moves in y this frame

💡 The cat barely rotated yet (-1.44°), so almost all movement goes right (x), with only a tiny downward nudge (y). This is correct — it is just the first of many frames.

Behaviour Across Frames

Each frame the cat gets more aligned and accelerates. Here is the pattern:

Frame state	angle_error	Velocities	What happens
Start	45° off	High ω , medium v	Rotating hard, drifting forward
Mid-turn	22° off	Medium ω , high v	Still turning, picking up speed
Aligned	$\sim 0^\circ$	$\omega \approx 0$, max v	Charging straight at mouse
Overshot	negative	ω corrects back	Small correction, stays fast
Caught!	0, dist $\approx$ 0	$v = 0$	Stop — mouse caught 🐭

Key Concepts

Why atan2 instead of atan?

`atan(dy/dx)` takes a single ratio and loses the individual signs of `dy` and `dx`. Two completely different directions can produce the same ratio:

Direction	dx	dy	Result
Up-right	+4	+4	<code>atan</code> → 45° ✓ <code>atan2</code> → 45° ✓
Down-left	-4	-4	<code>atan</code> → 45° ✗ <code>atan2</code> → -135° ✓
Up-left	-4	+4	<code>atan</code> → -45° ✗ <code>atan2</code> → 135° ✓
Straight up	0	+4	<code>atan</code> → crash ✨ <code>atan2</code> → 90° ✓

`atan2(dy, dx)` takes `dy` and `dx` separately, preserves both signs, and returns the correct angle for all 360°. It is the safe, unambiguous version.

atan = arctan = tan⁻¹

These are all the same thing, just different notations. `tan` gives you the ratio from an angle. `arctan` reverses it — given the ratio, it gives you back the angle.

```
tan(45°) = 1.0
arctan(1.0) = 45°    ← 'what angle has a tan of 1.0?'
```

What is dt?

`dt` (delta time) is the duration of one frame in seconds. Velocities are per-second values, so multiplying by `dt` scales them down to one frame's worth of movement.

Frame rate	dt
30 fps	dt = 0.033s
60 fps	dt = 0.016s
120 fps	dt = 0.008s

💡 Without dt , the simulation would run at different speeds on different machines. dt keeps movement consistent regardless of frame rate.

Angular vs Linear Velocity — why both?

Scenario	Result
Angular velocity ω only	Cat spins in place — position never changes
Linear velocity v only	Cat moves forward but can never steer
Both together	Cat turns and drives — reaches the mouse ✅

Think of it like a car: ω is the steering wheel, v is the gas pedal. You need both.

What are $k_angular$ and k_linear ?

They are tuning constants — personality dials with no deeper mathematical meaning. Adjust them until the movement feels right.

Setting	Behaviour
$k_angular$ (low)	Slow, lazy turns
$k_angular$ (high)	Snappy, sharp turns
k_linear (low)	Creeps toward mouse
k_linear (high)	Charges at mouse

💡 Typical starting values: $k_angular = 2.0 - 5.0$, $k_linear = 1.0 - 3.0$

What Happens If You Omit dt?

Your simulation becomes frame-rate dependent — the same code produces completely different behaviour on different machines.

Without dt

```
θ += ω           // adds degrees/second every frame — ignores how
long the frame was
cx += v × cos(θ)  // adds units/second every frame
```

Frame rate	What happens
30 fps	Runs 30 times/second → cat moves at 30× the per-second rate
60 fps	Runs 60 times/second → cat moves at 60× the per-second rate
Result	60fps machine moves cat TWICE as fast as 30fps — same code, different behaviour

With dt

```
θ += ω × dt
cx += v × cos(θ) × dt
```

```
30fps → dt = 0.033s → 30 × 0.033 = 1.0s worth of movement per second ✓
60fps → dt = 0.016s → 60 × 0.016 = 1.0s worth of movement per second ✓
```

Both machines produce identical cat movement. The math always adds up to exactly one second's worth regardless of frame rate.

💡 Think of it like speed in a car: '60 km/h' is meaningless without knowing how long you drove. *dt* is that time — one frame's worth of elapsed seconds.

The Feedback Loop — A Proportional Controller

The whole system is a feedback loop — also known as a Proportional Controller (P Controller). Each frame, the output (cat's position and heading) feeds back into the input of the next calculation.

The Loop Each Frame

Measure → Calculate Error → Correct → Repeat

1. Where is the mouse? → measure distance and angle
2. How far off am I? → angle_error, distance
3. Correct → apply ω and v
4. Repeat next frame → errors shrink each time

Why 'Proportional'?

Because the correction is always proportional to the error — bigger error, bigger correction:

```
 $\omega$  = k_angular × angle_error // big angle error → turn fast  
 $v$  = k_linear × distance // far away → move fast
```

As the cat closes in and aligns with the mouse, the errors shrink naturally — so the corrections shrink too. The cat slows and straightens on its own without any special stopping logic.

The Limitation — Oscillation

A pure P Controller can overshoot — the cat corrects too hard, passes the target, corrects again, and wobbles. This is why more advanced controllers exist:

Controller	What it adds
P (Proportional)	What we built — corrects based on current error
PD (Proportional-Derivative)	Adds damping — resists overshooting by accounting for how fast the error is changing
PID (P + Integral)	Adds memory of past error — corrects persistent drift that P and D miss

💡 *For a simple cat chasing a mouse, P is usually good enough. If you notice the cat wiggling around the target, adding a D term is the natural next step.*

Quick Reference — Formulas

What	Formula
Vector to target	$dx = mx - cx$ $dy = my - cy$ $dist = \sqrt{dx^2 + dy^2}$
Desired angle	$desired_angle = \text{atan2}(dy, dx)$
Angle error	$angle_error = desired_angle - \theta$ (wrap to $\pm 180^\circ$)

Angular velocity	$\omega = k_angular \times angle_error$
Linear velocity	$v = k_linear \times distance \times \cos(angle_error)$
Update heading	$\theta += \omega \times dt$
Update x position	$cx += v \times \cos(\theta) \times dt$
Update y position	$cy += v \times \sin(\theta) \times dt$